

SETU

सेतु • ಸೇತು

A Multilingual Voice & Text Bridge for English, Hindi, and Kannada

Project Report

Arnav Misra

July 2026

Table of Contents

1. Introduction.....	3
2. Problem Statement.....	3
3. Objectives.....	3
4. Proposed Solution.....	4
5. System Architecture.....	4
6. Modules Used.....	4
6.1 Frontend Module.....	4
6.2 Backend Module.....	5
6.3 Language AI Integration Module.....	5
6.4 Deployment & Configuration Module.....	5
7. Technology Stack.....	5
8. Key Features.....	6
9. Development Journey & Challenges.....	6
10. Limitations.....	6
11. Future Scope.....	7
12. Conclusion.....	7

1. Introduction

Most people who move between English and a language like Hindi or Kannada in daily life do it constantly and without much thought — in conversation, that is. The moment it needs to happen in text or over a recording, it stops being easy. Someone has to type the message, run it through a translator, read the result, and if a spoken version is needed too, find yet another tool for that. It works, but it is three or four steps for something that should be one.

Setu was built to cut that down to one step. Say or type something in English, Hindi, or Kannada, and get it back — written and spoken — in the other two. The name itself is the word for "bridge" in both Hindi and Kannada, which is really just what the tool tries to be: a short, direct path between three languages instead of a chain of separate apps.

2. Problem Statement

Translating between English, Hindi, and Kannada today usually means using three different kinds of tools one after another: something to turn speech into text, something else to translate that text, and a third tool if a spoken version of the result is needed. Each handoff between tools costs time, and each one is a place where something can get lost or typed wrong.

For someone comfortable with technology this is an annoyance. For a lot of people it is enough friction that they just don't bother, and either avoid the conversation or muddle through in whichever language is more convenient at the time, even if it isn't the one they'd rather use. That's a real cost in places where Hindi, Kannada, and English are all in everyday use side by side — homes, shops, schools, and any counter where the person on one side of it doesn't share a language with the person on the other.

What was missing was something simpler: one place to speak or type, in whichever language is natural in the moment, that hands back the other two languages immediately — both to read and to hear.

3. Objectives

Setu was scoped around a short list of things it had to do well, rather than a long list of things it could do eventually:

- Take input as either speech or typed text, in English, Hindi, or Kannada
- Translate that input into both of the other two languages, not just one
- Return each translation as both text and audio, since the two aren't always useful for the same purpose
- Stay simple enough that someone with no technical background can use it on a phone browser without instructions
- Never expose the API credentials that pay for the underlying AI service to anyone using the site
- Work as a link anyone can open, rather than something that has to be installed first

4. Proposed Solution

Setu splits the work between two pieces: a page that runs in the browser, and a small server behind it. The browser side is what the person actually sees and touches — recording a voice clip, typing a sentence, tapping play on a translation. The server side does the part that shouldn't happen on someone's phone: it talks to the language AI provider, keeps track of the API key that pays for that, and puts the results back together before sending them to the browser.

Splitting it this way solves two problems at once. It keeps the API key off the device entirely, so nobody using the site could ever extract it and misuse it, and it keeps the browser side light enough to run on an ordinary phone, since none of the actual language processing happens there.

The first version of this idea only worked one way — English in, Hindi and Kannada out — because that was the original request. Once it became clear that a Hindi or Kannada speaker would want the same thing in reverse, the logic was changed so that whichever language goes in, the other two always come out. That symmetry is now the whole point of the tool, not an add-on.

5. System Architecture

A single request moves through six steps, from the moment someone taps the microphone or hits "Bridge it" to the moment they see results on screen:

- The browser captures a voice recording or a typed sentence
- That input is sent to the server over a secure (HTTPS) connection
- If it was audio, the server sends it to the provider's Speech-to-Text service, which returns both a transcript and which language it detected
- For each of the two languages that still need to be produced, the server calls Translation, then Text-to-Speech, to get back translated text and a matching audio clip
- All of that — the original transcript and both translations, text and audio together — gets bundled into one reply
- The browser lays that reply out as cards: what was understood, and the two translations, each with a play button and a copy button

Sitting in front of all of this is a simple rate limit, capping how many requests any one visitor can make per minute. That's there for two reasons: it keeps a traffic spike from running up an unexpectedly large bill on the API, and it makes the audio endpoint — the most expensive one to call — harder to abuse once the site is public.

6. Modules Used

6.1 Frontend Module

The interface is plain HTML, CSS, and JavaScript — no framework. That was a deliberate choice: it loads in an instant, and there's no build step or dependency chain that could break on someone else's machine. It handles a few jobs directly:

- Two ways to give input — a Speak tab that uses the browser's own microphone recording, and a Type tab with a language picker
- Sending whatever was recorded or typed to the server, and showing a status message while it waits
- Laying out the response as cards — the recognised source text, then each translation with a play button for the audio and a copy button for the text
- A consistent visual language across the three cards, using a distinct colour per language so it's obvious at a glance which card is which

6.2 Backend Module

The server is written in Python using Flask, which is a small, unopinionated framework well suited to a project this size. It's responsible for:

- Serving the frontend files to the browser
- Two endpoints — one for typed text with a chosen source language, one for an uploaded audio recording
- Calling the language AI provider in the right order for each request and assembling the combined result
- Enforcing the per-visitor rate limit
- Reading the API key from a local configuration file that never leaves the server

6.3 Language AI Integration Module

None of the actual language understanding happens inside Setu's own code — that would be a project of its own. Instead, the backend calls out to Sarvam AI, a provider that specialises in Indian languages, using three of its services in sequence: Speech-to-Text to turn a recording into a transcript and identify which language was spoken, Translation to convert that text into a target language, and Text-to-Speech to turn the translated text back into spoken audio. Setu's own code is really just the logic that decides which of these three calls to make, in what order, for a given request.

6.4 Deployment & Configuration Module

A few smaller pieces round out the project so it's actually usable day to day rather than just runnable from a terminal:

- The API key and other settings live in a local `.env` file, kept out of any code that gets shared or version-controlled
- A one-time setup script and an everyday launch script, for both Windows and macOS/Linux, so starting the app doesn't mean typing commands every time
- Written instructions covering local setup as well as what to change before deploying the tool publicly

7. Technology Stack

Nothing here is exotic on purpose — the goal was to pick tools that are well understood and easy for someone else to pick up later, not to show off a stack:

Layer	Technology	Purpose
Frontend	HTML5, CSS3, JavaScript	User interface, audio recording, playback
Backend	Python, Flask	Request handling, orchestration, security
Rate limiting	Flask-Limiter	Per-visitor request throttling
Language AI	Sarvam AI (Speech-to-Text, Translation, Text-to-Speech APIs)	Core language intelligence
Configuration	python-dotenv	Secure loading of API credentials
Deployment target	Any Python-compatible web host (e.g. Render, Railway)	Public hosting over HTTPS

8. Key Features

- Works from either speech or typed text, in English, Hindi, or Kannada, and figures out which language was spoken on its own
- Always produces both of the other two languages, not a single translation
- Gives back text and audio for each translation, since sometimes you want to read it and sometimes you want to hear it
- Shows exactly what it heard from a voice recording, clearly labelled, before showing the translations — so it's obvious whether it actually understood you correctly
- Lets you copy any translated text or replay any audio clip with one tap
- Needs nothing installed — opens like any web page, and can be pinned to a phone's home screen if wanted
- Built to be shared as a link and used by many people at once, not just a personal script

9. Development Journey & Challenges

The tool didn't come together in a straight line, and a few of the turns along the way are worth recording, since they explain why it's built the way it is:

- Choosing a language provider: Bhashini, the Government of India's free platform for Indian languages, was the first choice, mainly because it's free with no usage cost. Signing up turned out to require an AISHE code — an identifier for registered academic institutions — which an independent developer simply doesn't have. Rather than use a code that didn't belong to the project, the build moved to Sarvam AI instead, which offers instant signup, a working free-credit allowance, and translation quality built specifically for Indian languages.
- Changing the backend language: the server was first written in Node.js. Partway through, it was rebuilt in Python instead, at request, without touching the frontend or changing how it talks to the language provider. That it was possible to swap the entire backend language without disturbing anything else is really just a sign that the two halves of the app were kept properly separate from the start.
- Making translation symmetric: the first version only handled English going out to Hindi and Kannada, and treated Hindi or Kannada input as English-only in return. That was corrected once it was clear the tool needed to work the same way regardless of which language someone starts in.
- Planning for public use: since the goal was always a tool other people could use, not just a personal script, things like the rate limit and the pre-launch checklist around spending caps and abuse prevention were built in early rather than bolted on after the fact.

10. Limitations

- The translation and speech quality depend entirely on Sarvam AI's models — Setu has no fallback if that service is slow or briefly unavailable
- Free credits cover testing and light personal use comfortably, but real public traffic will eventually cost money to run
- Only English, Hindi, and Kannada are supported; adding a fourth language means changes on both the frontend and backend, not just a configuration tweak
- Recognition accuracy can dip with heavy accents, background noise, or speech that mixes languages mid-sentence
- There's no login and nothing is saved between visits — each session starts fresh

11. Future Scope

- Add more Indian languages beyond the current three
- Keep a short history within a session so earlier translations don't disappear the moment you start a new one
- Let people download the generated audio clips instead of only playing them in-browser
- Add optional accounts for anyone who wants their usage or preferences remembered
- Add stronger abuse protection, such as a CAPTCHA on the audio endpoint, if usage grows enough to need it
- Look into a version that works with poor or no internet connection, using smaller on-device models

12. Conclusion

Setu started as a fairly narrow request — English going out to Hindi and Kannada — and became something a bit more complete along the way: a small, genuinely useful bridge between three languages that handles both directions and both text and speech. None of the individual pieces are complicated on their own; a browser recording audio, a small Python server, and a language API doing the real work. What made it worth building out properly was fixing the rough edges as they showed up — the blocked signup, the backend rewrite, the one-directional translation — rather than leaving them as known issues. The result is a tool simple enough to hand to someone with no technical background and just say: speak, or type, and read or listen to the rest.